● B3.2 Fundamentals of OOP for multiple classes

●.1 Explain and apply the concept of inheritance in OOP to promote code reusability

Syllabus point: B3.2.1

Inheritance expresses an **"is-a"** relationship. A subclass is a more specialized version of a parent class. The subclass automatically **reuses** non-private attributes and methods from the parent and can **add** new behaviour or **override** inherited methods.

Inheritance promotes code **reuse** because common behaviour can be written once in the parent class and then inherited by many subclasses.

●.1.1 Python syntax

Python

```python
class Person:
    def __init__(self, name: str):
        self.name = name

    def introduce(self) -> str:
        return f"Hello, I am {self.name}."

class Teacher(Person):
    def __init__(self, name: str, subject: str):
        super().__init__(name)
        self.subject = subject

    def introduce(self) -> str:
        # Reuses Person's introduce() string and appends the subject details
        return f"{super().introduce()} I teach {self.subject}."

class Student(Person):
    def __init__(self, name: str, grade_level: int):
        super().__init__(name)
        self.grade_level = grade_level

    def introduce(self) -> str:
        # Reuses Person's introduce() string and appends the grade details
        return f"{super().introduce()} I am in grade {self.grade_level}."
```

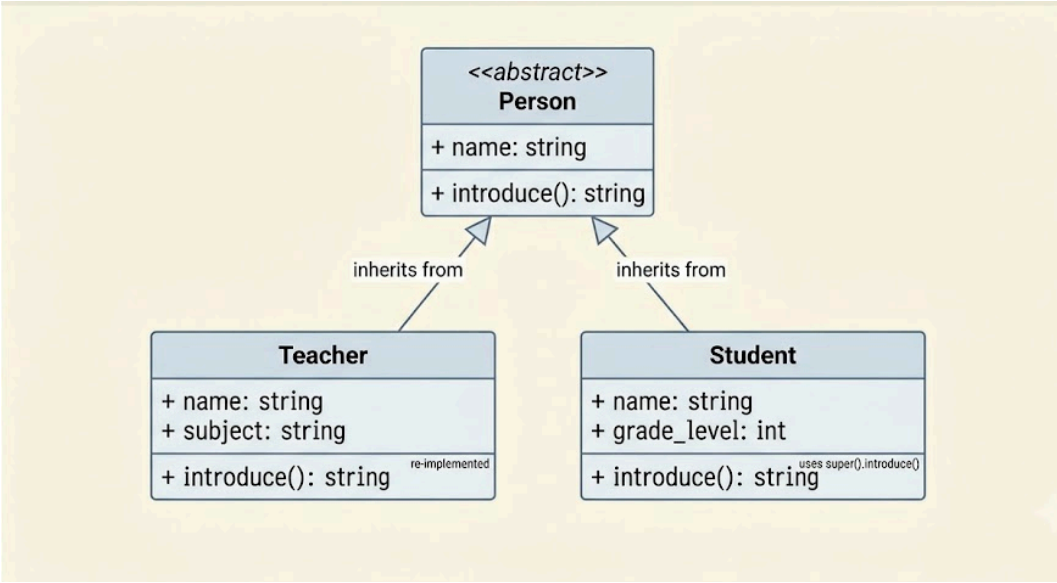
●.1.2 Important explanation

- `Teacher` and `Student` inherit from `Person`, so they reuse the `name` attribute and the 'introduce' method.
- `super().__init__(name)` calls the parent initializer so the inherited part of the object is set up correctly.
- The statement super().introduce() invokes the logic defined in the Person class. Each subclass uses **method overriding** to provide specific, tailored functionality for its own instances.
- The subclass can add new data, such as `subject`, and can also override methods.

The attached syllabus mentions access modifiers. In Python, there are no Java-style `private`, `protected`, or default modifiers. The closest explanation is that public attributes are openly accessible, single-underscore names are internal by convention, and double-underscore names trigger name mangling rather than true private access.

Advantages of inheritance:

- **Reusability**: A subclass can reuse the data and behaviors defined in its superclass.
- **Extensibility**: A subclass can add its own behaviors and data to extend its superclass's functionality.
- **Easy to maintain**: Changes made to the superclass are automatically reflected in the subclass, reducing maintenance overhead.
- **Simplifies testing:** Inherited functionality from the parent class does not need to be retested in the subclass.



●.1 Construct code to model polymorphism and its various forms, such as method overriding

Syllabus point: B3.2.2

The word polymorphism comes from Greek:

- poly = many
- morph = forms

So polymorphism literally means "many forms." Think of a dog. A Chihuahua, a Golden Retriever, and a German Shepherd all look different and behave somewhat differently, but they are all still **dogs**. If you say: "Fetch!", each dog responds to the same command, but may carry it out differently.

- A Chihuahua may run quickly and return.
- A Golden Retriever may fetch enthusiastically.
- A stubborn dog may take its time.

The **same command** produces **different behaviors** depending on the specific dog.

This is the basic idea of polymorphism: **one interface, many forms of behavior.**

Polymorphism in OOP means that the same method call or interface can work with objects of different types. This makes code more flexible because client code can focus on what an object can do rather than on its exact class.

●.1.1 Dynamic polymorphism in Python

Dynamic polymorphism happens when different subclasses provide their own versions of the same method and the program chooses the correct version at runtime. In Python, this is usually demonstrated through method overriding.

Python

```
class Shape:
    def area(self) -> float:
        raise NotImplementedError

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius: float):
```

```
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

def print_area(shape: Shape) -> None:
    print(shape.area())
```

The function `print_area()` does not need to know whether it receives a Rectangle or a Circle. It simply calls `.area()`. The specific implementation used depends on the actual object passed in.

Dynamic binding enables the creation of code that is not dependent on a specific type. The method **area()** can accept an object of any **Shape** subclass as a parameter, whether it be a circle, square, triangle, or any other shape that has not yet been defined. This means that the **Client** code (print_area() in this case) is decoupled from **Shape's** specific subclasses and only interacts with **Shape**. As a result, the **Client** class remains unaffected by new types of **Shape** that may be introduced in the future.

Advantages of dynamic polymorphism

- Polymorphism allows an external program to use the same method on all subclasses;
- By allowing overridden methods in child classes to add only the code that is needed for the unique processing of that sub-class;
- Dynamic Binding allows you to write code that is not dependent on a specific type

Dynamic polymorphism requires runtime method resolution. During compilation, the compiler can determine that shape.area() is a valid call because the reference type Shape defines an area() method. However, if area() is overridden by subclasses, the compiler cannot know which implementation will be executed because the actual object type may only be known at runtime. Therefore, the method call is resolved during

program execution, where the runtime system determines the actual object type and invokes the appropriate method implementation. It is worth noting that this dynamic dispatch may incur a small runtime overhead due to the additional method lookup. The term dynamic polymorphism refers to the fact that method binding is performed at run time rather than at compile time.

Override common methods:

Overriding common built-in methods—often referred to as dunder (double-underscore) or magic methods—allows custom objects to integrate naturally with Python's built-in operators, functions, and behaviors.

```python
class Item:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

class ShoppingCart:
    def __init__(self, owner: str):
        self.owner = owner
        self.items: list[Item] = []

    def add_item(self, item: Item):
        self.items.append(item)

    # 1. Override string representation for end users
    def __str__(self) -> str:
        return f"{self.owner}'s Cart ({len(self.items)} items)"

    # 2. Override formal representation for developers/debugging
    def __repr__(self) -> str:
        return f"ShoppingCart(owner={self.owner!r},
items_count={len(self.items)})"

    # 3. Override length behavior so len(cart) returns the item count
    def __len__(self) -> int:
        return len(self.items)

    # 4. Override equality comparison to compare owners rather than memory address
    def __eq__(self, other: object) -> bool:
        if isinstance(other, ShoppingCart):
            return self.owner == other.owner
        return False

# --- Usage ---
```

```
cart1 = ShoppingCart("Alice")
cart1.add_item(Item("Book", 12.99))
cart1.add_item(Item("Pen", 1.50))

cart2 = ShoppingCart("Alice")

# Demonstrating overridden methods:
print(str(cart1))      # Calls __str__  -> Output: Alice's Cart (2 items)
print(repr(cart1))     # Calls __repr__ -> Output: ShoppingCart(owner='Alice',
items_count=2)
print(len(cart1))      # Calls __len__  -> Output: 2
print(cart1 == cart2)  # Calls __eq__   -> Output: True
```

It is worth noting that it is common that this is referred to as operator overloading as it defines how a built-in operator (like +, ==, or <) behaves when applied to your custom objects. For example, by defining **__eq__** (which overrides the base method), you are overloading the **==** operator. Operator overloading allows Python's built-in operators to work seamlessly with your custom objects.

●.1.2 What about static polymorphism?

In Java, static polymorphism often refers to method overloading. Python does not support traditional compile-time method overloading by signature. True static polymorphism determines method execution at **compile time** rather than relying on runtime resolution.

Java

```
class Calculator {
	int add(int a, int b) {
	return a + b;
	}
	double add(double a, double b) {
	return a + b;
	}
	int add(int a, int b, int c) {
	return a + b + c;
	}
}
public class Main {
	public static void main(String[] args) {
	Calculator calc = new Calculator();
	System.out.println(calc.add(2, 3));     // add(int, int)
	System.out.println(calc.add(2.5, 3.5));   // add(double, double)
	System.out.println(calc.add(1, 2, 3));  // add(int, int, int)
	}
}
```

In the above example, the compiler can determine which add() method to invoke based on the number and types of arguments used in the method call. The term static polymorphism refers to the fact that method binding is performed at compile time rather than at runtime.

- static polymorphism (compile-time polymorphism) can **significantly maximize code efficiency**, particularly when compared to dynamic polymorphism (runtime polymorphism). Because resolution happens at compile time rather than runtime, static polymorphism eliminates execution overhead and opens up major compiler optimizations. It is worth noting that runtime performance overhead specifically describes the extra lookup duration runtime polymorphism spends determining which method implementation to call.
- **Improved readability**: You can use the same function name for conceptually similar operations, instead of inventing multiple names.
- **Ease of use**: The compiler automatically selects the correct version based on argument types, so the programmer doesn't need to remember different names.
- **Code organization**: Keeps related functionality grouped under one identifier, reducing clutter in your codebase.

●.2 Explain the concept of abstraction in OOP

Syllabus point: B3.2.3

Abstraction means focusing on the essential features of an object while hiding unnecessary implementation detail. An abstract class describes what related subclasses must be able to do, without fully defining how every method works.

In Python, abstraction is commonly modelled using the `abc` module and abstract base classes. An abstract class cannot be instantiated until its abstract methods are implemented by a concrete subclass.

Modular programming is a methodology for designing software that emphasizes breaking down the functionality of a program into distinct and interchangeable modules. Each module should be self-contained and have all the necessary components to execute a specific aspect of the desired functionality.

The PaymentMethod class and its subclasses form a cohesive module because they all serve the same functionality. The parent class defines the abstract method pay(), which each concrete subclass must implement. **This abstraction allows users to focus on the pay() interface without worrying about implementation details**. By applying the factory design pattern, users don't even need to know which subclass is instantiated—the factory selects the appropriate payment method. **This makes the component easy to use and reduces system complexity. At the same time, the design is extensible -** new payment methods can be added without affecting other parts of the system, since the concrete classes remain decoupled from external

modules. The system can switch to a new payment method with minimal changes to the system.

Abstraction is a cornerstone of modular software engineering. By suppressing unnecessary execution details and exposing only essential interfaces, **abstraction serves as the primary mechanism for dividing a complex software system into independent, manageable code fragments.**

●.2.1 Python example using `abc`

Python

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        pass

class CardPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Paid {amount} by card")

class CashPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Paid {amount} in cash")
```

The abstract class `PaymentMethod` defines a common interface: every payment method must implement `pay()`. This helps create modular code because the rest of the program can work with the abstract idea of a payment method rather than with one concrete class.

In Python, **ABC** stands for **Abstract Base Class**. Provided by the built-in `abc` module, an ABC defines a blueprint for other classes. It allows you to specify a set of methods and properties that any subclass **must** implement before it can be instantiated.

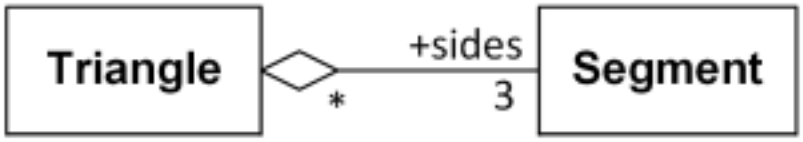
- **Cannot be Instantiated Directly:** You cannot create an object directly from an ABC. Trying to instantiate it raises a `TypeError`.
- **Enforces Method Implementation:** If a subclass inherits from an ABC, it must override and implement every method marked with the `@abstractmethod` decorator.
- **Defines Interfaces:** It establishes a contract across different classes, ensuring consistent behavior across your codebase.

●.3 Explain the role of composition and aggregation in class relationships

Syllabus point: B3.2.4

Both composition and aggregation are "has-a" relationships between classes, but they are not equally strong.

- Aggregation: It represents a type of relationship between classes in which the part can exist independently of the whole. It is a distinct type of association that involves a relationship between two classes, much like regular association. However, it is a one-way, directional association that differs from regular association. It represents a **has-a** relationship.



  - "Triangle has a 'sides' collection of **three** line Segments.
  - Each line Segment could be part of none, one, or several (*) triangles."

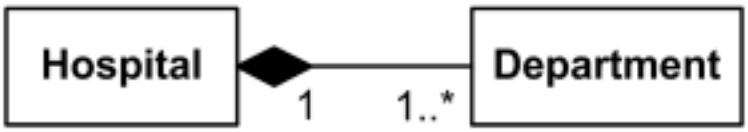
- Composition: It represents a type of relationship between classes in which the part is strongly tied to the whole and usually does not meaningfully exist without it. It represents a relationship between a whole and its parts. If the whole component is removed, typically all of its individual parts are also removed. The representation of composition in a diagram is a binary association that features a black diamond filled with color at the aggregate end.

Aggregation implies that the subcomponents can function independently of the aggregating class, while in composition, the subcomponents are tightly coupled and cannot exist outside the aggregating class.



- "Folder could contain **many (*)** files, while each File has exactly **one** Folder parent.
- If Folder is deleted, all contained Files are deleted as well.
- The "no sharing" rule is the key to composition."[1]

[1] K. Fakhroutdinov, "UML composition," UML graphical notation overview, examples, and reference., https://www.uml-diagrams.org/composition.html (accessed May 16, 2023).



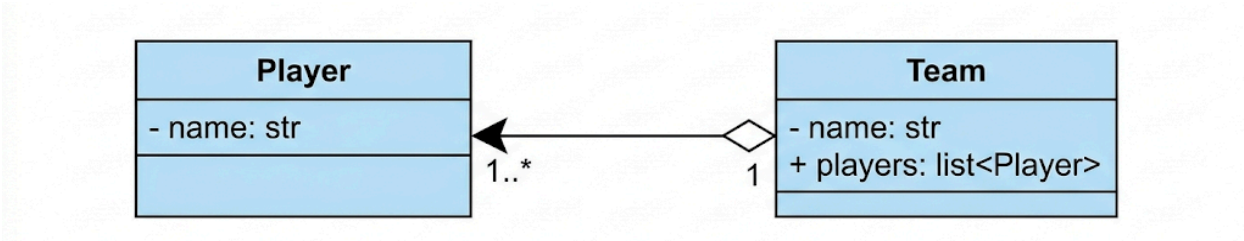
- "Hospital has **1 or more** Departments, and each Department belongs to exactly **one** Hospital.
- If the Hospital is closed, so are all of its Departments."[2]

●.3.1 Aggregation example

Python

```
class Player:
    def __init__(self, name: str):
        self.name = name

class Team:
    # the players are created outside this class and are passed as a parameter.
    def __init__(self, name: str, players: list[Player]):
        self.name = name
        self.players = player
```



A Player can exist without a particular Team, so this is aggregation. The team groups players, but it does not define their existence.

Advantages of aggregation/composition:

- Aggregation allows code to be reused. The code defined in the aggregated class can be reused in the aggregating class.
- Aggregation allows for better organization of the objects as the variables and methods of the aggregated data will be kept in the class to which they belong.

---

[2]K. Fakhroutdinov, "UML composition," UML graphical notation overview, examples, and reference., https://www.uml-diagrams.org/composition.html (accessed May 16, 2023).

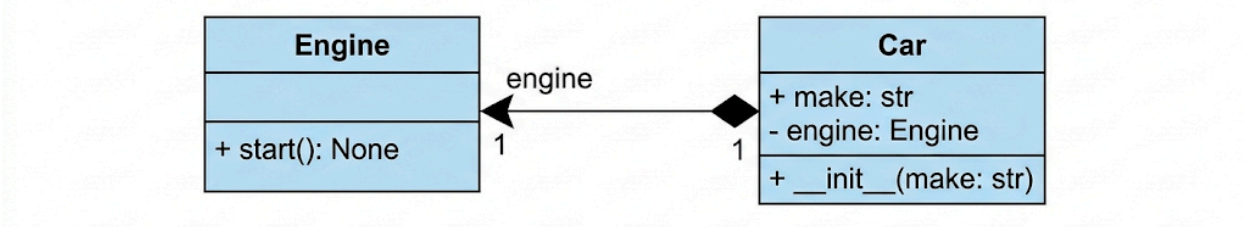
Disadvantages of aggregation/composition:

- Using aggregation can lead to more complex code when accessing functions of an aggregated object.
- Aggregation increases dependencies. A change in one class could have an unexpected consequence for another class.

●.3.2 Composition example

```python
class Engine:
    def start(self) -> None:
        print("Engine started")

class Car:
    def __init__(self, make: str):
        self.make = make
        self.engine = Engine() # create a new engine
```



Here, the Car creates and owns its Engine. This is composition because the engine is being treated as an internal part of the car object.

●.3.3 Why the distinction matters

This distinction helps when designing object lifecycles and responsibilities. If a smaller object should be created, stored, and destroyed as part of a larger object, composition is often the clearer model. If the smaller object should continue to exist independently, aggregation is usually a better description.

2.4.4 Summary

Composition (strong “part-of”)

• The whole creates the part.

• The part cannot exist without the whole.

• Lifetime is tied. In composition, the part object is almost always created inside the composing object, because the whole controls the part's lifetime.

Aggregation (weak "has-a")

• The whole receives the part from outside.

• The part can exist independently.

• The aggregate does not own the part.

• Lifetime is not tied. In aggregation, the part object is usually created outside the aggregating object, then passed in (injected) because the part has an independent lifecycle.

●.4 Explain commonly used design patterns in OOP

Syllabus point: B3.2.5

A design pattern is a reusable design idea for a recurring programming problem. A design pattern is not a complete program; it is a common structure that can be adapted to many situations.

The syllabus explicitly names singleton, factory, and observer. These patterns were not covered in the old revision note, so the explanations below are expanded with reputable external sources.

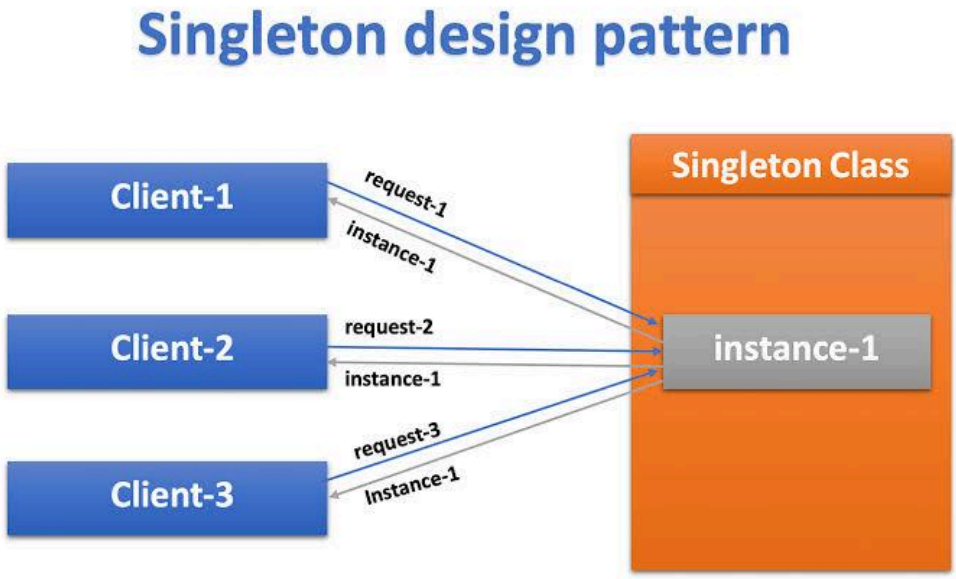
●.4.1 Singleton

The Singleton Pattern ensures that a class has only one instance while providing a global point of access to that instance. It is primarily used when multiple parts of an application need to share a single, coordinated resource or central state.

Singleton ensures that only one instance of a class exists and gives the program a single shared access point to that instance. This can be useful for shared configuration, logging setup, or application-wide services.

- **Logging Services:** Loggers write application events to a single destination (like a file or stdout). Using a singleton prevents file-access race conditions or out-of-order log entries from competing threads.
- **Application Configuration & Settings:** App-wide properties (such as environment variables or feature flags) need to be loaded once at startup and read globally without re-parsing files.

However, Singleton should be used carefully. A single shared object can make code harder to test and can introduce hidden global state.



Python

```python
class Settings:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return
        self.theme = "dark"
        self._initialized = True
```

- Main idea: only one instance is created.
- Typical use: shared settings or a single service object.
- Risk: hidden global state can reduce modularity.

Key Benefits

- **Controlled & Single Resource Access:** Prevents conflicting access to shared resources that can only handle one operation at a time (e.g., database connection pools, file loggers, print spoolers, or hardware devices).
- **Memory and Overhead Efficiency:** Prevents performance degradation by instantiating expensive objects (like configuration parsers or heavy service

connections) once rather than repeatedly allocating and garbage-collecting memory.
- **Lazy Initialization:** Unlike static classes or global variables that initialize as soon as the application launches, a Singleton can be configured to instantiate only when it is accessed for the first time.
- **Guaranteed Single State Across Modules:** Ensures all modules, threads, and services in your application read from and write to the exact same shared data (e.g., application runtime settings or in-memory caches).

●.4.2 Factory

A factory pattern centralizes object creation. Instead of client code directly instantiating every concrete class, one function or class decides which concrete object to create.

This is helpful when the exact class depends on user input, configuration, or some runtime condition. It also reduces coupling because the rest of the program can work with a common interface rather than depending on many concrete classes.

Primary Uses

- **Dynamic Object Creation at Runtime:** When you don't know ahead of time which exact types or dependencies your application will need (e.g., choosing a payment gateway like Stripe, PayPal, or Square based on user selection).
- **Encapsulating Complex Setup:** When creating an object requires multi-step configuration, reading environment variables, or setting up database handles before it can be used.
- **Decoupling Frameworks and Libraries:** When building extensible tools or plugins where downstream developers need to supply custom implementations without modifying core framework code.

Key Benefits

- **Loose Coupling:** Client code depends on an abstract base class or interface (like `BaseNotifier`) rather than concrete classes (`EmailNotifier`, `SMSNotifier`). Changing an implementation details class won't break the caller.
- **Open/Closed Principle (OCP):** You can introduce new product types (e.g., adding a `WhatsAppNotifier`) without altering existing client code—you simply extend the factory or register the new class.
- **Single Responsibility Principle (SRP):** Centralizes object creation logic into one dedicated place, keeping business logic clean and unburdened by initialization code.
- **Consistent Object State:** Ensures objects are always fully configured and valid before they reach the rest of your application.

Python

```python
from abc import ABC, abstractmethod

# Abstract Base Class enforcing the 'send' contract
class BaseNotifier(ABC):

    @abstractmethod
    def send(self, message: str) -> None:
        """All concrete services MUST implement this method."""
        pass


# Concrete Implementation 1
class EmailNotifier(BaseNotifier):
    def send(self, message: str) -> None:
        print(f"[EMAIL] Sending: {message}")


# Concrete Implementation 2
class SMSNotifier(BaseNotifier):
    def send(self, message: str) -> None:
        print(f"[SMS] Sending: {message}")


# Concrete Implementation 3
class PushNotifier(BaseNotifier):
    def send(self, message: str) -> None:
        print(f"[PUSH] Sending: {message}")



class NotifierFactory:
    _notifiers = {
        "email": EmailNotifier,
        "sms": SMSNotifier,
        "push": PushNotifier,
    }

    @classmethod
    def get_notifier(cls, channel_type: str) -> BaseNotifier:
        notifier_cls = cls._notifiers.get(channel_type.lower())

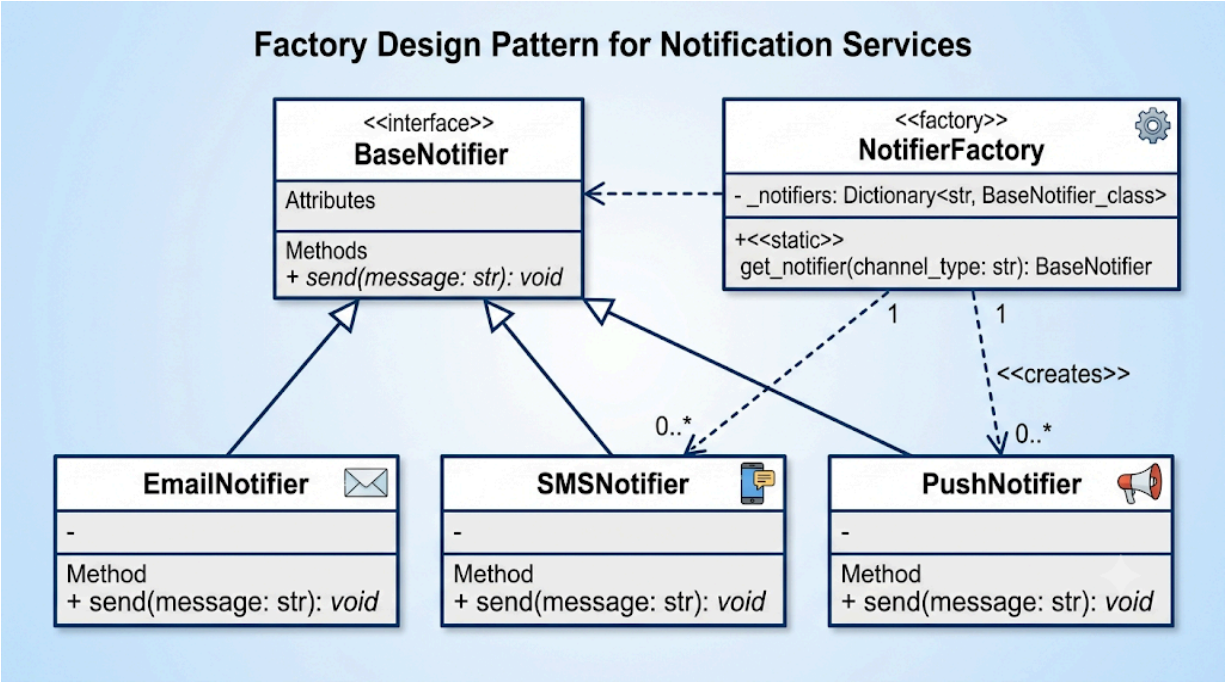
        if not notifier_cls:
            raise ValueError(f"Unknown notification channel: '{channel_type}'")

        # Returns a NEW instance of the requested class
        return notifier_cls()
```

```python
# 1. Dynamically create objects based on runtime input
notifier1 = NotifierFactory.get_notifier("email")
notifier1.send("Your order has shipped!")  # Output: [EMAIL] Sending: Your order has shipped!

notifier2 = NotifierFactory.get_notifier("sms")
notifier2.send("Your OTP code is 4921")    # Output: [SMS] Sending: Your OTP code is 4921

# 2. Factory returns distinct instances (Not a Singleton)
n1 = NotifierFactory.get_notifier("email")
n2 = NotifierFactory.get_notifier("email")
print(n1 is n2)  # Output: False (New instance created every time)
```



- Main idea: creation is separated from use.
- Typical use: selecting one of several concrete classes at runtime.

●.4.3 Observer

Observer is used when one object needs to notify several other objects about a change in state. It supports event-driven thinking: one subject changes, and its observers react.

This is useful in interfaces, messaging systems, and applications where updates should be broadcast to multiple listeners without tightly coupling the classes together.

Python

```python
from abc import ABC, abstractmethod

# --- Abstract Subscriber Contract ---
class Observer(ABC):
    @abstractmethod
    def update(self, temperature: float) -> None:
        """Called by the Subject when its state changes."""
        pass


# --- Abstract Publisher Contract ---
class Subject(ABC):
    @abstractmethod
    def attach(self, observer: Observer) -> None:
        pass

    @abstractmethod
    def detach(self, observer: Observer) -> None:
        pass

    @abstractmethod
    def notify(self) -> None:
        pass


# --- Concrete Subject ---
class WeatherStation(Subject):
    def __init__(self):
        self._observers: list[Observer] = []
        self._temperature: float = 0.0

    def attach(self, observer: Observer) -> None:
        if observer not in self._observers:
            self._observers.append(observer)

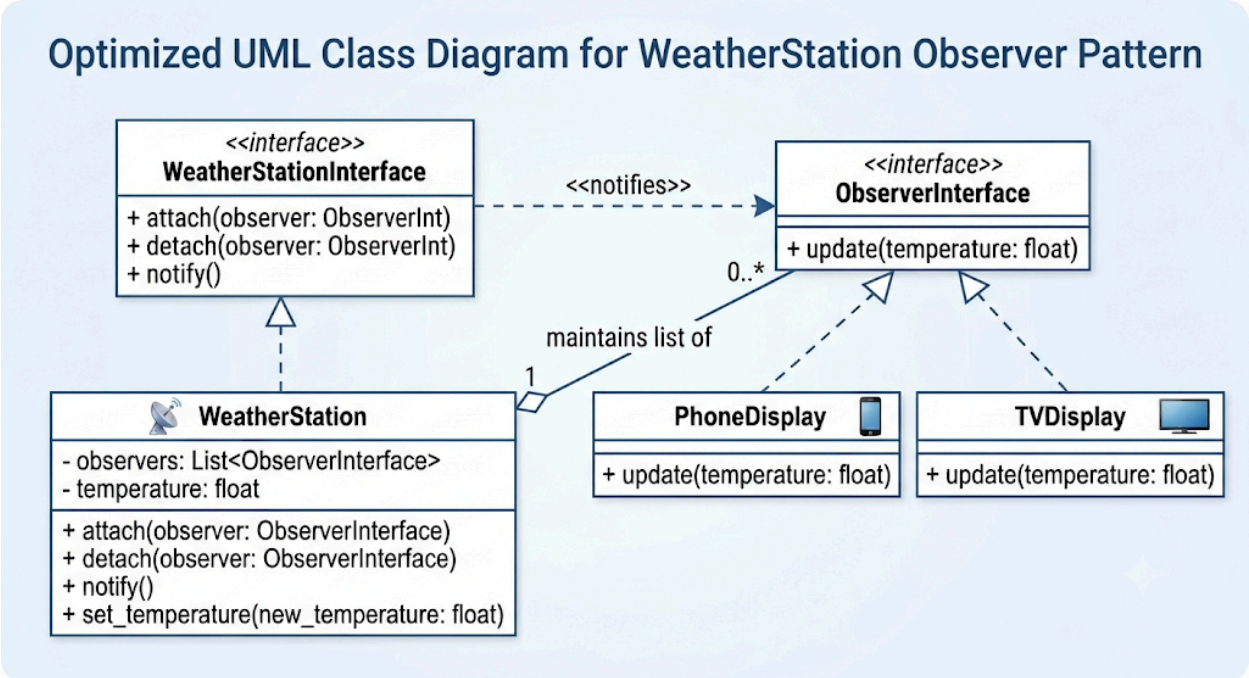
    def detach(self, observer: Observer) -> None:
        self._observers.remove(observer)

    def notify(self) -> None:
        for observer in self._observers:
            observer.update(self._temperature)

    def set_temperature(self, new_temperature: float) -> None:
        print(f"\n[WeatherStation] Temperature updated to {new_temperature}°C")
        self._temperature = new_temperature
        self.notify()
```

```python
# --- Concrete Observers ---
class PhoneDisplay(Observer):
    def update(self, temperature: float) -> None:
        print(f"[Phone Display] Showing updated temp: {temperature}°C")


class TVDisplay(Observer):
    def update(self, temperature: float) -> None:
        print(f"[TV Display] Weather Alert: Temp is now {temperature}°C")
```



- Main idea: observers subscribe, then receive updates when the subject changes.
- Typical use: event systems, notifications, and interfaces.

Primary Use Cases

- **Event-Driven Systems & GUIs:** Updating user interface components automatically when underlying data models change (e.g., MVC architectures where the View observes the Model).
- **Real-Time Data Feeds:** Broadcasting live updates to multiple listeners, such as stock ticker updates, sports scoreboards, or weather monitoring dashboards.

- **Notification and Messaging Services:** Triggering downstream actions after a specific event occurs (e.g., sending an email, logging an audit event, and updating an analytics dashboard when a user places an order).

Key Benefits

- **Loose Coupling:** The Subject only knows that its observers implement a specific interface (`ObserverInterface`). It does not need to know their concrete classes, internal state, or implementation details.
- **Support for Dynamic Relationships:** Observers can be attached or detached at runtime based on application state or user preferences.
- **Broadcast Communication:** The Subject automatically transmits state updates to any number of subscribers in a single operation rather than requiring manual, individual method calls for every dependent object.